Day 2: SELECT, FROM, Aliases and DISTINCT

Phase 1: Foundations | Day 2 of 30 | about 60-90 minutes

Before you start

Open your query tool: the **Query Tool** in pgAdmin, or the **Execute SQL** tab in DB Browser for SQLite, connected to the **sqlcourse** database.

Quick check: run `SELECT * FROM customers;` You should see **8 rows**. If the table is missing or empty, re-run the load script from Day 1 before continuing.

Learning objectives

By the end of today you will be able to:

- Read and write a `SELECT ... FROM` query and explain what each part does.
- Pick specific columns, in the order you want, instead of relying on `SELECT *`.
- Build calculated columns and rename columns and tables with **aliases**.
- Use `DISTINCT` to remove duplicate rows, on one column or several.

Our dataset (from Day 1)

Table	Columns
`customers`	`customer_id (PK), name, email, city, signup_date`
`products`	`product_id (PK), name, category, price`
`orders`	`order_id (PK), customer_id (FK), order_date, status`
`order_items`	`order_id (FK), product_id (FK), quantity, unit_price`

Part 1: Concept (15 min)

1. The shape of a query

Every query you write today has the same shape: say **what** you want, then say **where** it lives.

```
SELECT  <the columns you want>
FROM    <the table they live in>;
```

Think of a table as a spreadsheet. `FROM` picks the sheet, and `SELECT` picks which columns to show. The database sends back a new, temporary table called the **result set**. A `SELECT` only *reads* data. It never changes the original tables, so you can experiment freely.

A few ground rules:

- Keywords are not case-sensitive (`select` works), but writing them in **UPPERCASE** makes queries easier to read. We will follow that convention.
- Spaces and line breaks are free. Put `FROM` on its own line so queries stay readable as they grow.
- A semicolon `;` marks the end of a statement. Always include it.

- Lines starting with `--` are comments. The database ignores them, so use them to leave notes.

2. All columns versus chosen columns

`SELECT *` means "every column, in the table's own order". It is perfect for a first look at a table. In real work, name the columns you need instead: you control the order, the output stays small, and your query does not break when someone adds a column later. List columns separated by commas, with **no comma after the last one**.

3. Calculated columns (expressions)

A column in your `SELECT` list does not have to be a stored column. You can calculate on the fly with `+ - * /`, and join text with `||`. The calculation appears in the result only. Nothing is saved back to the table.

> **Careful with NULL**
>
> If any part of a calculation is **NULL**, the whole result is NULL. For example, `'abc' || NULL` gives NULL, not `'abc'`. Customer 7 in our data has no email, which makes a perfect demonstration in the exercises.

4. Aliases: giving things a nicer name

An **alias** is a temporary name, set with `AS`. It affects only the output of that one query.

- **Column alias:** `price * 1.18 AS price_with_tax` labels a calculated column.
- **Quoting:** if the alias contains spaces or capital letters you want to keep, wrap it in **double quotes**: `AS "List Price"`. **Single quotes** are only for text values like `'Delhi'`.
- **Table alias:** `FROM customers AS c` lets you write `c.name` instead of `customers.name`. It looks optional today, but it is essential once we join tables on Day 13.

PostgreSQL note: unquoted names are folded to lowercase, so `AS Total` shows up as `total`. Use double quotes to keep capitals.

5. DISTINCT: removing duplicate rows

Imagine a list of customers with their cities. Delhi might appear many times. Put `DISTINCT` right after `SELECT` and the database returns each unique row only once.

```
SELECT DISTINCT city
FROM customers;
```

- **DISTINCT looks at the whole row you selected.** With `SELECT DISTINCT a, b` you get each unique *combination* of a and b.
- It is written **once**, right after `SELECT`, and applies to all listed columns. Writing `DISTINCT(city), name` does *not* apply it to city alone; the brackets change nothing.
- All NULLs count as **one** value, so NULL appears at most once in a DISTINCT result.
- It is the quickest way to answer "what different values exist in this column?"

Part 2: Guided practice (20 min)

Type each query yourself (do not copy and paste, your fingers learn too). Run it, then read the **Notice** line before moving on.

Step 1: Look at a whole table

```
SELECT * FROM customers;
```

Notice: SELECT * returns every column and row. Count the rows: 8. Look at the column order, it matches the table definition. Repeat with `products`, `orders` and `order_items` to get a feel for all four tables.

Step 2: Choose columns

```
SELECT name, email FROM customers;
```

Notice: Only two columns come back. Now swap the order and run again: `SELECT email, name FROM customers;` The columns follow *your* order, not the table's. Customer 7 shows an empty/NULL email.

Step 3: A calculated column with an alias

```
SELECT name, price, price * 1.18 AS price_with_tax
FROM products;
```

Notice: A new column appears, labelled price_with_tax. Delete the `AS price_with_tax` part and run again. Your database invents an ugly label. Aliases fix that.

Step 4: Aliases with spaces

```
SELECT name AS "Product Name", price AS "List Price"
FROM products;
```

Notice: The double quotes keep the capital letters and the space. Try changing them to single quotes and observe the error.

Step 5: Joining text

```
SELECT name || ' lives in ' || city AS sentence
FROM customers;
```

Notice: Each row becomes a readable sentence. The text in single quotes is a **literal**, the same in every row. If any customer had a NULL city, that row would show NULL.

Step 6: A table alias

```
SELECT c.name, c.city
FROM customers AS c;
```

Notice: The result looks identical to `SELECT name, city FROM customers`. The alias c is just shorthand that we will rely on heavily with joins.

Step 7: Unique values

```
SELECT DISTINCT city
FROM customers;
```

Notice: Compare with `SELECT city FROM customers;` which returns 8 rows. DISTINCT returns fewer, because repeated cities collapse into one.

Step 8: DISTINCT on two columns

```
SELECT DISTINCT customer_id, status
FROM orders;
```

Notice: Now you get unique *pairs* of customer and status. A customer with two orders of the same status appears once; with two different statuses, twice.

Step 9: DISTINCT and NULL

```
SELECT DISTINCT email
FROM customers;
```

Notice: The NULL email appears exactly once, in among the real emails. DISTINCT treats NULL as a value to deduplicate.

Step 10: No table needed

```
SELECT 'Hello, SQL' AS greeting, 2 + 3 AS total;
```

Notice: SELECT can run without FROM when you only need literals and arithmetic. Handy as a calculator. It works in both PostgreSQL and SQLite.

Part 3: Exercises (25-40 min)

Write each query yourself before checking the answer key at the end. Each exercise has a **Check** line so you can verify your own result.

Exercise 1 (easy)

Show every column and every row from the **products** table.

Check: 8 rows; product_id runs from 101 to 108.

Exercise 2 (easy)

From **customers**, show only the **name** and **city** columns.

Check: 8 rows, 2 columns.

Exercise 3 (easy)

From **products**, show the name, category and price, but relabel the columns **product_name**, **category** and **list_price**.

Check: 8 rows; the headings read product_name, category, list_price.

Exercise 4 (medium)

Which cities do our customers live in? Return each city **once**.

Check: Fewer than 8 rows, and no city repeated.

Exercise 5 (medium)

List the different values that appear in the **status** column of **orders**.

Check: One row per distinct status; far fewer than the 8 orders.

Exercise 6 (medium)

From **order_items**, show order_id, product_id and a calculated column **line_total** (quantity multiplied by unit_price).

Check: 12 rows, 3 columns.

Exercise 7 (medium-hard)

From **customers**, create one column called **contact** that looks like `Name <email>`, with the email wrapped in angle brackets. Which row is NULL, and why?

Check: 8 rows. Exactly one row is NULL: think about customer 7.

Exercise 8 (hard)

Using a **table alias** o for orders, list the customer_ids that have placed at least one order, each shown once, with the column named **customer_with_orders**.

Check: At most 6 rows. Customers 7 and 8 must *not* appear (they have no orders). Why can the answer never reach 8 rows, given there are 8 orders?

Stretch goal

1. Run `SELECT DISTINCT order_id, product_id FROM order_items;` How many rows do you get? Before you run it, predict the number and explain your reasoning using the primary key of `order_items`.

2. Predict which returns *more or equal* rows, then test: `SELECT DISTINCT customer_id FROM orders;` or `SELECT DISTINCT customer_id, status FROM orders;` Why?

3. Run `SELECT name city FROM customers;` (no comma). It does not fail. What did it return, and what does this tell you about the optional `AS`?

Recap (5 min)

One-line takeaway

`SELECT` chooses the columns, `FROM` chooses the table, aliases rename things in the output, and `DISTINCT` removes duplicate rows.

Common mistakes

1. A trailing comma before FROM. `SELECT name, FROM customers` is a syntax error. No comma after the last column.

2. A missing comma between columns. `SELECT name city` silently treats `city` as an alias for `name`. You get one mislabelled column and no error message.

3. Single quotes for aliases. Use double quotes for names with spaces. Single quotes are for text values.

4. Thinking DISTINCT works on one column of many. It deduplicates the whole selected row. `DISTINCT(city), name` is not different from `DISTINCT city, name`.

5. Relying on SELECT * everywhere. It is great for exploring, but name your columns in anything you plan to keep.

6. Forgetting NULL spreads. One NULL inside `||` or arithmetic makes the whole result NULL.

Coming up on Day 3: filtering rows with `WHERE`, comparison operators, and combining conditions with `AND`, `OR` and `NOT`. Today you chose which *columns* to see; tomorrow you choose which *rows*.

Answer key with explanations

Your result may use different spacing or capitalization of keywords, and that is fine. What matters is the same rows and columns. Row counts below are the ones guaranteed by the course dataset; other values depend on the data you loaded on Day 1, so compare against your own output.

Exercise 1

```
SELECT *
FROM products;
```

Expected: 8 rows, 4 columns (product_id, name, category, price).

Why: The star means "all columns". There is no comma and no column list, just the table name after FROM.

Exercise 2

```
SELECT name, city
FROM customers;
```

Expected: 8 rows, 2 columns.

Why: Naming columns controls exactly what you see. The order you write them is the order you get.

Exercise 3

```
SELECT name AS product_name,
       category,
       price AS list_price
FROM products;
```

Expected: 8 rows, headings product_name, category, list_price.

Why: Each alias follows its column with AS. The category column keeps its original name because it needs no alias. Aliases rename only the output; the table still calls the columns name and price.

Exercise 4

```
SELECT DISTINCT city
FROM customers;
```

Expected: Each city exactly once, so fewer than 8 rows.

Why: DISTINCT sits directly after SELECT. Without it you would get 8 rows, one per customer, with cities repeated.

Exercise 5

```
SELECT DISTINCT status
FROM orders;
```

Expected: One row per different order status.

Why: This is the standard way to discover which values a column holds. Statuses such as shipped or pending repeat across many orders, so DISTINCT collapses them.

Exercise 6

```
SELECT order_id,
       product_id,
       quantity * unit_price AS line_total
FROM order_items;
```

Expected: 12 rows, 3 columns, one per order line.

Why: quantity * unit_price is evaluated for every row. Without the alias, your database would give the column a generic name. The result is a **line total**; adding these up per order is something you will do with SUM on Day 8.

Exercise 7

```
SELECT name || ' <' || email || '>' AS contact
FROM customers;
```

Expected: 8 rows; the row for customer 7 is NULL.

Why: || joins text pieces. Customer 7's email is NULL, and anything joined with NULL becomes NULL, so the whole contact string disappears for that row. Fixing this properly needs COALESCE, which you will learn on Day 22.

Exercise 8

```
SELECT DISTINCT o.customer_id AS customer_with_orders
FROM orders AS o;
```

Expected: At most 6 rows; customers 7 and 8 absent.

Why: The alias o is declared in FROM as orders AS o, so the column can be written o.customer_id. DISTINCT removes repeat customers. 8 orders cannot come from more than 6 customers (7 and 8 never ordered), so at least one customer ordered more than once, which means the result is always smaller than 8 rows.

Stretch goal answers

1. Exactly **12 rows**, the same as the table itself. The primary key of order_items is the pair (order_id, product_id), so every pair is already unique and DISTINCT has nothing to remove.

2. `SELECT DISTINCT customer_id, status FROM orders;` returns more or equal rows. It keeps each unique *pair*, so a customer with orders in two different statuses counts twice. Adding columns to a DISTINCT list can only keep the count the same or increase it.

3. It returns a single column holding the customer names, but labelled **city**. Without a comma, SQL reads the second word as an alias for the first, because AS is optional. This is why the missing-comma mistake is so sneaky: no error, just wrong output.

Common-mistake demo: this query fails with a syntax error because of the trailing comma:

```
SELECT name,
       email,
FROM customers;
```

End of Day 2. Great work, see you on Day 3!